

Buildathon 2026 Participant Guide

This is your operating guide for the Bengaluru Tech Week Buildathon. Read it before arrival, keep it open during the build, and use the final checklist before you submit. The submission deadline is 3:00 PM IST.

Your day

Time	What happens	What you should do
8:00–9:00	Breakfast, check-in and setup	Arrive early. Confirm your team, laptop, power, GitHub access, development tools and optional Databricks Free Edition access.
9:00–12:00	Kickoff and build session	Choose one Entire track, fork only after the official start, configure the required Entire workflow, build a narrow end-to-end slice and create your early checkpoints.
12:00–1:00	The Noon Curveball and lunch	Stop, preserve your last stable version, close the current agent session, receive the new constraint, start fresh and reconstruct the project from checkpoint context before editing.
1:00–3:00	Build session and submission	Implement the constraint, test the changed behavior, verify the Entire Graph evidence, create the final checkpoint, finish BUILDATHON.md and submit before 3:00 PM.
3:00–5:00	Judging and winner announcement	Keep the project runnable. Mentors complete first-level review; eight finalists present; judges deliberate and awards are announced.

What makes this Buildathon different

You are not being asked to produce a generic app around a prompt. You must build a useful developer product on one of three [Entire](#) open-source surfaces, preserve decision context through checkpoints, use structural evidence from the Entire Graph where required, and respond to an additional non-negotiable constraint revealed at noon.

Teams may also make [Databricks](#) essential to the same product and opt in to the separate Best Use of Databricks awards. There remains one project and one submission; a team may win both an Entire main-challenge prize and a Databricks award.

Before you arrive

Bring and verify

- A laptop, charger and any adapters you need. Bring a power bank only as a backup; do not rely on it for the full day.
- A working GitHub account with permission to create forks and push to your fork.
- Your preferred coding agent and local development tools already installed and signed in.
- A local runtime that can run tests and a browser or API client that can demonstrate the product.
- A [Databricks](#) Free Edition account if your team wants to compete for Best Use of Databricks. Create and open it before the event so account issues do not consume your build window.
- A fallback demo asset: a recent screenshot or short screen recording for any step that depends on a live service.

Allowed before 9:00 AM

Research, user discovery, problem framing, sketches and planning are allowed before the official start. Implementation must begin after 9:00 AM on 6 September. Do not arrive with a prebuilt solution, completed feature branch or reusable implementation that gives your team an unfair head start.

Team readiness

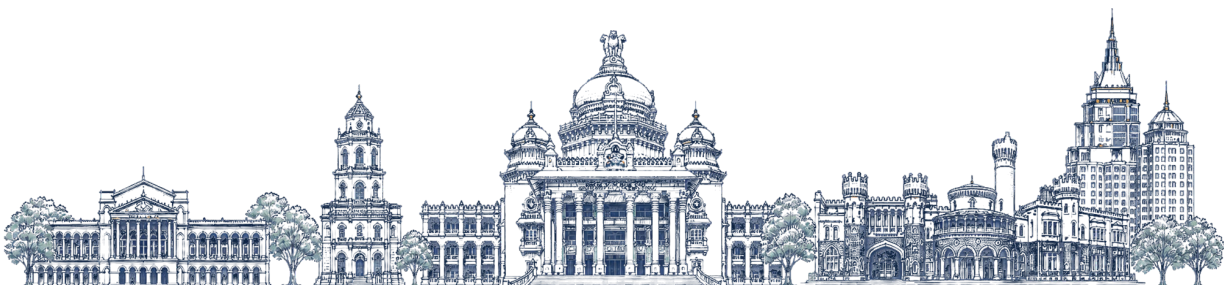
- Name one person as submission owner. This person checks every link and submits before 3:00 PM.
- Name one person as the final demo owner. They should be able to explain the system even if another teammate wrote most of the code.
- If using [Databricks](#), choose one final demo workspace and one deployment owner. Use Git for shared code; never store secrets in the repository.
- Agree how you will record decisions, failures and verified outcomes in checkpoints and BUILDATHON.md.

Account links

[Create or open Databricks Free Edition](#)

[Buildathon details and updates](#)

[Bengaluru Tech Week Code of Conduct](#)



Choose one Entire innovation track

Your team must select one track. The project should solve a real developer or agent-workflow problem and make [Entire](#) essential to the solution. Adding Entire only for tracking, without using its context, checkpoints or graph evidence to improve the product, will not qualify.

Track 1 — Build a Checkpoint-Native Developer Experience

Build a developer tool or workflow powered by context preserved in [Entire](#) Checkpoints. Git shows what changed; checkpoints should preserve why a change happened, what the agent attempted, which assumptions were made, what failed and what remains unresolved.

- Review an implementation against its stated intent.
- Identify unfinished requirements or unresolved risks.
- Create a change-risk or release-readiness report.
- Hand work to another developer or agent without losing the original reasoning.
- Resume an agent-assisted workflow with confidence after a break.

Success looks like this: checkpoint context is an essential input, and a developer or agent completes a useful task more accurately or efficiently than they could from a Git diff alone.

Track 2 — Build with Graph Intelligence

Use the [Entire](#) Graph's structural view of a codebase to support a useful decision or action. The graph can surface definitions, call relationships, types, routes and semantic changes. Your product must turn that evidence into a workflow that a developer or agent can inspect and verify.

- Impact-aware code review or change-risk analysis.
- Test selection based on affected relationships.
- Migration planning or dependency-aware refactoring.
- Codebase onboarding and guided exploration.
- Combining Entire Graph findings with checkpoint intent.

Raw graph output is not enough. Your product should produce a useful decision, recommendation or workflow, show where the evidence came from, and let the user verify it against source code or tests.

Track 3 — Bring Entire to a New Agent or Workflow

Extend or create an [Entire](#) integration for a coding agent, editor, development environment, CI process or software-delivery workflow. The integration must capture meaningful development context and keep working in its intended operating mode.

- Support for a new coding agent or editor workflow.
- A CI or pull-request workflow that uses checkpoint context.
- A multi-agent coordination workflow.
- A protocol adapter between Entire and another developer tool.

External SDKs and fixtures are allowed, but the submitted implementation must live in the designated Entire fork. Calling an Entire command from another interface is not sufficient by itself.

Required Entire workflow

Complete these steps after the 9:00 AM kickoff. All implementation must happen in the clone created through the [Entire](#) mirror workflow.

1. Fork your selected repository

Fork the repository associated with your chosen track only after the Buildathon begins. Use existing issues for inspiration, or propose an original feature, integration or workflow.

2. Sign in and create the mirror

```
entire login
```

```
entire repo mirror create
```

```
entire repo clone /gh/YOUR-GITHUB-HANDLE/REPOSITORY
```

Select your fork and the India region when prompted. Clone through Entire, then work inside that clone.

3. Enable checkpoints

```
entire enable -y --agent YOUR-BUILD-AGENT
```

```
entire status
```

Commit meaningful milestones so code and context remain connected. Your checkpoint history must include at least:

- Initial understanding and intended architecture.
- The last stable state before the Noon Curveball.
- Your response to the Noon Curveball.
- Final implementation and verification.

Checkpoint quality matters more than quantity. Capture decisions, rejected options, failures, assumptions, open risks and the evidence that changed your approach.

4. Activate Entire Graph

```
entire plugin install graph
```

```
entire graph version
```

```
entire graph init-agents --repo .
```

Start a fresh agent session after activation so the agent receives the graph instructions. During development, show at least:

- A graph search or definition lookup.
- A relationship or impact analysis before a high-risk change.
- A final semantic diff analysis of the submitted implementation.

Graph results are evidence, not an oracle. Verify findings against source code, tests and runtime behavior. Never present incomplete or uncertain graph output as a fact.

The Noon Curveball

At exactly 12:00 noon, every team receives an additional track-specific constraint that becomes mandatory. The first version is the proof of what you built. The second version is the proof that your process can absorb change without losing intent.

11:45 AM — preserve the stable state

- Stop adding optional features and get the product back to a runnable state.
- Commit the last stable version.
- Create the required pre-noon Entire Checkpoint.
- Record the current intent, architecture, completed work, unresolved work and important technical risks.

12:00 PM — receive the constraint

- Stop implementation and close the current coding-agent session.
- Receive the official Curveball from the organising team. Do not exchange cards or use another track's instruction.
- Start a fresh agent session and use checkpoint context to reconstruct the project before editing.
- Run Entire Graph impact analysis before changing the affected area.
- Implement the smallest complete response, test it and record the result in a new checkpoint.

What judges will look for

A strong response preserves the product's core idea, changes a real architectural or reliability decision, uses checkpoint context accurately, uses graph evidence where relevant, remains testable and explains why the revised behavior can be trusted.

The Curveball's specific instructions are intentionally not published in advance. Mentors and organisers must not disclose them before noon.

Competing for Best Use of Databricks

[Databricks](#) is an additional award category within the same project.

Free Edition setup

- Each teammate may explore in an individual [Databricks](#) Free Edition workspace.
- Choose one final workspace for the shared dataset, deployed app or endpoint and final demonstration.
- Keep notebooks, app code, configuration and documentation in the shared Git repository. Never commit access tokens or credentials.
- Name one deployment owner and test that the owner can sign in and open every required resource before judging.

Good scope for a one-day build

- An AI app or agent using Databricks Apps, hosted foundation models, model serving or AI Search where available.
- An analytics experience using notebooks, SQL, dashboards or Genie to answer a concrete question.
- A data pipeline that ingests, cleans and transforms representative data into a useful result.
- A data-backed application using Lakehouse data and Databricks Apps for the user experience.
- An ML experiment that uses a reproducible dataset, experiment tracking and an evaluation result.

Free Edition constraints to design around

Area	Planning constraint
Service model	No-cost, serverless, quota-limited environment for non-commercial learning and prototyping; no production SLA.
SQL and jobs	One serverless 2X-Small SQL warehouse; up to five concurrent job tasks per account.
Pipelines	One active pipeline per supported pipeline type.
Models and AI Search	Endpoint, throughput and model availability limits apply; no GPU or provisioned throughput; AI Search is limited.
Apps and database	Up to three Apps, with idle auto-stop behavior; one Lakebase project where available.
Workspace	One workspace and one metastore per account; several account-level and enterprise controls are unavailable.
Quota	When quota is exhausted, affected compute may be unavailable for the rest of the day or month. Build and test your narrow path early.

Work locally when practical, deploy one narrow slice early and preserve a screenshot or recording of any fragile live step. If live infrastructure fails during judging, explain the expected behavior, show the prepared evidence and continue with the product story.

[Databricks Free Edition overview](#)

[Current Databricks Free Edition limitations](#)

Responsible use of AI and data

- Do not upload confidential company information, customer data, passwords, private keys or personal data that you do not have permission to use.
- Do not put secrets in prompts, source repositories, screenshots, checkpoint context or BUILDATHON.md.
- Verify generated code, SQL, citations and outputs. Test the cases that matter to your user and disclose what remains unverified.
- Explain your data sources, permitted use, sampling or transformation, and important limitations.
- Clearly label synthetic, cached, mocked or prototype data and responses.
- You remain accountable for the implementation, even when a coding agent created it. Every teammate presenting should understand the relevant code and product decisions.

Final 20-minute checklist

- The final commit is pushed and the SHA matches the submission.
- The project launches from a clean checkout or the documented setup path.
- Required checkpoints open and clearly explain the four milestones.
- [Entire](#) Graph evidence and the final semantic-diff analysis are recorded.
- BUILDATHON.md is complete, readable and free of secrets.
- Tests covering the critical and Curveball behavior pass.
- [Databricks](#) resource links and data notes are included if you opted in.
- The demo owner can sign in, open every resource and run the critical path.
- A fallback screenshot or recording is locally available.
- The submission is sent before 3:00 PM IST.

Code of Conduct and safety

All participants must follow the Bengaluru Tech Week Code of Conduct. Treat teammates, organisers, mentors, judges, venue staff and other participants with respect. Report harassment, unsafe behavior, accessibility barriers or urgent welfare issues to the event desk. For non-urgent questions, write to hello@bengalurutechweek.com.

[Read the Bengaluru Tech Week Code of Conduct](#)

Submission package

Submit one project before 3:00 PM IST. Late or incomplete submissions may not reach judging. Open every link in a private or signed-out browser where appropriate, and make sure judges have the access they need without receiving a private credential.

Required submission fields

- Selected [Entire](#) track.
- GitHub fork URL and final commit SHA.
- Entire mirror or project URL.
- Links to the required Entire Checkpoints.
- Clear setup, run and test instructions.
- Working product demonstration or a reliable fallback recording.
- A complete BUILDATHON.md file in the repository root.
- If opting into Best Use of [Databricks](#): the Databricks capabilities used, why they are essential, workspace/app/endpoint/demo URL, relevant repository paths, reproduction steps, data provenance and how the Curveball affected the Databricks workflow.

BULDATHON.md outline

```
# Project name
## One-sentence summary
### Problem, intended user and why it matters
## Selected Entire track and why Entire is essential
## Architecture and main workflow
## Entire Graph findings and verification
## Noon Curveball: what changed and how we adapted
### Checkpoint links and what each checkpoint proves
## Setup, run and test instructions
## Databricks use, data sources and limitations (if applicable)
## Known limitations and next steps
```

Final demo readiness

- State the user and problem in one sentence.
- Show the working product and the critical path, rather than a slide-only walkthrough.
- Explain why [Entire](#) is essential to the solution.
- Show one useful checkpoint and one graph finding that changed or verified a decision.
- Explain the Noon Curveball, the behavior that changed and the test that proves it.
- If opting into [Databricks](#), show the essential Databricks function and the evidence that it is working.
- Close with known limitations and the next step toward production readiness.

How projects are judged

Entire main challenge — 100 points

Criterion	Points	What strong evidence shows
Problem and innovation	20	Clear user, specific problem, useful new capability and a real use of Entire .
Technical implementation	25	Working end-to-end workflow, sensible architecture, important behaviors tested and failures handled safely.
Response to the Curveball	15	Constraint addressed completely; assumptions revisited; response implemented, tested and explained.
Use of Entire Checkpoints	15	Checkpoints preserve useful intent and decision context; the fresh session can resume from them.
Use of Entire Graph	15	Graph evidence identifies relevant code or impact; findings are verified against source and tests.
Demonstration and future potential	10	Clear, reproducible demo; limits acknowledged; credible next step.

Best Use of Databricks — 100 points

Criterion	Points	What strong evidence shows
Meaningful use	30	Databricks is essential to a core workflow and materially improves what the product can do.
Working implementation and reliability	25	Judges can reproduce or inspect the path; failure handling and fallback evidence are credible.
User value and product decisions	20	Specific user need, focused scope and sensible trade-offs.
Data quality, provenance and responsible use	15	Permitted data, traceable transformations, disclosed limits and safe handling.
Curveball response	10	The Databricks-backed workflow adapts to the new constraint and the changed behavior is verified.